The Analysis Exercises [1]

The Analysis Exercises and how they're assessed

The first four exercise sheets contain a short "Analysis Exercise" which will ask you to

1. write C++ code that performs specified tasks,

2. explore the results and, where appropriate, perform some accompanying analysis,

3. summarise the results (and the analysis) concisely in a "powerpoint"-style presentation.

The tasks will be assessed on GRADESCOPE. You will have to upload (i) your C++ source code and (ii) the pdf file of your presentation to the course's GRADESCOPE page, accessible via Canvas. To ensure conciseness, the pdf file will have a page limit.

GRADESCOPE will then perform some automated tests on your code, allocate grades and provide diagnostics. Before the deadline you can upload the files as many times as you want, and thus deal with any problems that are reported. The final grade for the exercise will be based on the most recent submission received before the deadline.

The code and the report will be subject to a light-touch review, the spirit being very much like the first-year undergraduate tutorials: we primarily want to see that you've had a go at the question, and will briefly look at the source code and the pdf file. If it contains a sensible attempt you'll get full marks; if things look sloppy you'll get half marks; if there's no file (or it's just some dummy file) you'll get *nul point* as the Germans don't say.

The main aim of our looking at the files is to get a feeling for common errors and to spot room for improvement. We will briefly discuss our own "model" solution and contrast it with what we've seen in the students' submissions in the following computer lab class. It's important to stress that there's no one right way to do things – this applies to writing code and reports! Some of it is a matter of style, but there may be ways to do things objectively better.

While these exercises don't carry many marks (see below), they act as a "warm up" for the two bigger projects (worth 30% and 60% respectively) to be done later. These projects will be assessed in essentially the same way, but with more attention paid to the report (and more specific instructions given). Hence the main reward for engaging with these exercises is that you will have a much better feeling for what is expected. If you want any more detailed comments on your code and report, please ask during the lab classes.

To give you some guidance, the Canvas page provides the model solutions (C++ code, the pdf file for the report, and the latex source code used to generate it) for an extended version of a question from Exercise Sheet 1. Have a look and experiment with uploading the files to GRADESCOPE (though you obviously won't get any marks for this!).

Analysis Exercise 0: The Fibonacci sequence

1. Write a C++ code `analysis_exercise0.cpp` that computes the first 16 Fi-

[1] Version 2026/09/24.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

2026/09/24, 09:49:00

bonacci numbers f_n ($n = 0..15$), defined by the sequence

$$f_0 = 1, \tag{1}$$
$$f_1 = 1, \tag{2}$$
$$f_n = f_{n-1} + f_{n-2} \quad \text{for} \quad n \geq 2. \tag{3}$$

Think carefully about how to perform one iteration of this sequence within a loop. You do *not* need to store all the previous values! What's the minimum number of values you need to keep track of as the iteration proceeds?

2. Plot f_n as a function of n.

3. Validate your code by checking that the computed numbers actually satisfy the specified relation, and by comparing them against the explicit expression for the Fibonacci numbers, given, e.g., on Wikipedia at

`https://en.wikipedia.org/wiki/Fibonacci_sequence`

The final version of your code should output (on the screen, using `std::cout`) 16 lines, each containing three numbers, namely: the value of n; the value of f_n computed by the iteration; and the value of f_n computed by the explicit expression, so the first few lines should look like this:

```
0 1 1
1 1 1
2 2 2
3 3 3
4 5 5
5 8 8
```

(And, yes, we will check that you're not just outputting the same number twice!)

Upload your well-documented driver code, `analysis_exercise0.cpp` and the pdf file containing your results, `analysis_exercise0.pdf` (which must have **no more than five pages** incl. the title page which must show your name and student ID) to the course's GRADESCOPE page, accessible via Canvas.
You can upload your files as many times as you want. The grade for the exercise (out of a maximum of zero!) will be based on the final version received before the submission deadline of February 31, 3024.

Grades will be awarded based on the following criteria:

1. GRADESCOPE**-based assessment of code:**

   Up to 2 marks: Does the code produce the required results?

2. **Assessment of coding:**

   2 marks: Clear, well-documented and efficient code (no unnecessary steps or wasteful memory usage) with sensibly chosen variable/function names. Good, clear layout.

   1 mark: Code suffers from a significant number of these: inefficient (structure and memory usage), badly organised, unintuitive variable names, poor or inconsistent

2026/09/24, 09:49:00

comments.

0 marks: No code, or code that is a contender for the obfuscated C++ contest[2]. Code that doesn't even compile, or code that produces the results but not by the requested method.

3. **Assessment of report:**

2 marks: Concise report providing all the requested information; good layout and font size; clear and well-annotated, self-explanatory plots (where plots were requested)

1 mark: Report suffers from a significant number of these: doesn't provide all the requested information; poor layout; too much information (particularly if this is achieved by using an overly small font size); poor plots (in terms of layout, annotation and readability).

0 marks: No report or a report that suffers from any of these: doesn't respect the page limit; doesn't address the questions and/or is hard to follow; extremely poor (or no) plots (where plots were requested).

Each Analysis Exercise is worth 2.5% of the overall grade for the course. However, the main intention is for these exercises to be "formative", i.e. they're supposed to give you practice, and to help you identify areas you can improve before working on the "big" projects. You should therefore compare what you've done with our model solutions (published shorty after the submission deadline for each Analysis Exercise) and ask for advice and more detailed feedback during the computer lab classes.

[2]Sadly, that doesn't exist, but there is a C version: `https://www.ioccc.org/`.

2026/09/24, 09:49:00